Lean through Artform Calculus

An introduction to Lean, read beside Chapter 1 of the book

Part I explains the basics of Lean, a programming language in which proofs are checked by the computer. Part II writes the constructions of Chapter 1 of *Artform Calculus* in Lean. References such as Section 1.4, Figure 1.9 or Definition 1.5 point into the book; their numbers are those of its 52nd edition (29 September 2026). Every listing is cut from two Lean files, which Lean 4.34.1 accepts as a whole.

Contents

Part I. The basics of Lean **3**

Running Lean 3

Values and types 4

Definitions and functions 5

Definitions by cases, and recursion 6

Inductive types 7

Structures 8

Types that depend on values 8

Propositions are types 9
Tactics 10
Arguments that Lean finds 12
Reading Lean's messages 12

Part II. Chapter 1 of the book in Lean **14**
From the book's notation to Lean 14
Rods: their rules and their laws 15
Witnesses, and three equalities in $2 + 3 = 5$ 17
Laws as scripts 18
A tagged choice and a complete assignment 19
Formation lines 20
Time 22
Meaning 25
Spaces and states 27
Scripts and roles 28
What Lean does not see 30
Every formation line in Lean 30

Bibliography **32**

Part I. The basics of Lean

Lean is a programming language in which programs and proofs are written in one notation and checked by one program. A Lean file is a list of declarations. Lean reads the file from top to bottom, and for each declaration it either accepts it or says what is wrong with it. This part teaches enough of the language to read Part II, which writes the constructions of Chapter 1 of the book in Lean. It assumes no experience of programming.

Every example was checked by Lean 4.34.1. A comment after `--` at the end of a line shows what Lean answers to that line. Where Lean refuses a line, its message follows the listing, as Lean prints it.

Running Lean

The quickest way to start needs nothing installed. The Lean playground at `https://live.lean-lang.org` is an editor in the browser: type on the left, and Lean answers on the right. For longer work, install the editor VS Code and its official Lean 4 extension. The extension installs Lean itself and checks a file while it is being typed. Lean's answers appear in a panel beside the text, called the *infoview*, and an error is underlined in red where it occurs. Lean also runs in a terminal: `lean Basics.lean` checks the file `Basics.lean` and prints the answers and the errors.

The examples of this guide are in the folder `textbook/tutorial` of the repository that holds it. Part I's are in `LeanBasics.lean` and Part II's in `ArtformChapter1.lean`. Those files check every answer quoted here, so an editor shows nothing while the answers are right. The folder `play` holds copies without the checks, which show every answer in the infoview. To try them, copy `play` to a folder outside the repository and open it in VS Code; inside the repository, the editor would first set up the whole project, Mathlib included.

The examples use Lean's core language only. Most mathematicians who use Lean also use Mathlib, a large library of mathematics written in Lean; nothing here needs it.

Values and types

Every expression in Lean has a type. Two commands ask about an expression: `#check` shows its type, and `#eval` computes its value.

```
#eval 2 + 3                 -- 5
#check 2 + 3                -- 2 + 3 : Nat
#eval "glass " ++ "bead"    -- "glass bead"
#check (2, "two")           -- (2, "two") : Nat × String
#check [1, 2, 3]            -- [1, 2, 3] : List Nat
#eval [1, 2, 3].length      -- 3
```

The colon is read *has type*, as in the book: `2 + 3 : Nat` says that $2 + 3$ is a natural number. `Nat` is the type of the natural numbers, `String` of texts, `Nat × String` of pairs of a number and a text, and `List Nat` of lists of numbers. A text is written between double quotes, and `++` joins two texts.

Numbers need care. `Nat` has no negative numbers, so subtraction stops at zero, and division of natural numbers drops the remainder, which `%` gives. To compute in another type, write the expression

with its type, `(e : T)`. On the integers `Int` subtraction goes below zero, and on the rationals `Rat` division is exact:

```
#eval 7 - 10              -- 0
#eval (7 - 10 : Int)      -- -3
#eval 7 / 2               -- 3
#eval 7 % 2               -- 1
#eval (1/4 + 1/3 : Rat)   -- (7 : Rat)/12
```

The last line is the sum $\frac{1}{4} + \frac{1}{3} = \frac{7}{12}$ of Figure 1.16. Lean prints a rational with its type.

Definitions and functions

`def` gives a name to a value. The name comes first; then the inputs, each in parentheses with its type; then a colon and the type of the result; and then `:=` and the value.

```
def double (n : Nat) : Nat := 2 * n
#eval double 21           -- 42
#eval double (double 3)   -- 12
#check double             -- double (n : Nat) : Nat
```

A function is applied by writing it before its input, without parentheses: `double 21`. Parentheses only group, as in `double (double 3)`. `#check` of a defined name shows its inputs and the type of its result.

A function is a value, and `fun` builds one without a name: `fun n => 2 * n` is the function $n \mapsto 2n$, of type `Nat → Nat`. A function of two inputs takes them one at a time, so `add 2` is again a function: the one that adds two to its input.

```
#check fun n : Nat => 2 * n  -- fun n => 2 * n : Nat→Nat
def add (m n : Nat) : Nat := m + n
#check add 2                 -- add 2 : Nat→Nat
def twice {α : Type} (f : α→α) (x : α) : α := f (f x)
#eval twice double 3         -- 12
```

The first input of `twice` is a type, α. The braces make it *implicit*: Lean works out α from the other inputs, here `Nat`, and the caller does not write it. `twice double 3` doubles 3 twice.

Definitions by cases, and recursion

A definition may be given case by case, with one line for each form the input can take. A natural number is either 0 or $n+1$ for a smaller number n, and on the line for $n+1$ the definition may use itself at n:

```
def sumTo : Nat→Nat
  | 0     => 0
  | n + 1 => (n + 1) + sumTo n
#eval sumTo 4  -- 10
```

So `sumTo 4` is $4+3+2+1+0 = 10$, the ten points of the tetractys (Section 1.6). Lean accepts such a definition only after two checks: every case is covered, and every call of `sumTo` inside it is at a smaller number, so that the computation ends. A definition that leaves a case out is refused:

```
def isZero : Nat→Bool
  | 0 => true
```

Lean refuses it:

```
Missing cases:
(Nat.succ _)
```

The message names the case that was left out: a number of the form $n+1$, which Lean writes as `Nat.succ _`.

Inductive types

A new type is made by listing the ways to build its values. Each line names a *constructor* and says what it takes. A colour is light or dark, and a tree is either a leaf carrying a letter or a node joining two trees:

```
inductive Colour where
  | light
  | dark
inductive Tree where
  | leaf : Char→Tree
  | node : Tree→Tree→Tree
```

Every value of the type is built by its constructors, and only by them; Section 1.11 calls them the introduction rules. A function on a type is defined by cases, one for each constructor, and on a node it may use itself on the two smaller trees. The tree built from a, b and c below has three leaves, as the construction trees of Section 1.4 have one leaf for each atom.

```
def Tree.size : Tree→Nat
  | .leaf _   => 1
  | .node s t => s.size + t.size
def abc : Tree := .node (.leaf 'a') (.node (.leaf 'b') (.leaf 'c'))
#eval abc.size  -- 3
```

Two pieces of notation appear here. `Tree.size` is the function `size` belonging to the type `Tree`, and for a tree `t`, `t.size` means `Tree.size t`. Where Lean already knows that a tree is meant, `.leaf` means `Tree.leaf`. The natural numbers are an inductive type too, with the two constructors `zero` and `succ`, and the cases `0` and `n + 1` of `sumTo` are these two.

Structures

A *structure* is a type of records with named fields. A value gives its fields either by name or in order between angle brackets, and a field is read with a dot:

```
structure Interval where
  lo : Nat
  hi : Nat
def i : Interval := { lo := 2, hi := 5 }
def j : Interval := ⟨2, 5⟩
#eval i.hi - i.lo  -- 3
```

A structure is an inductive type with one constructor, which takes all the fields, and `⟨2, 5⟩` applies that constructor.

Types that depend on values

Types are values too: `Nat` has type `Type`. So a function may return a type, and a type may depend on a number. `Fin n` is the type of the numbers below n, the cells of the book's rod R_n. A value of it is a number together with a proof that the number is below n, written between angle brackets. The words `by decide` ask Lean to find the proof itself, by computing.

```
#eval (⟨2, by decide⟩ : Fin 5)  -- 2
#eval (7 : Fin 5)               -- 2
```

The second line shows a trap. A bare numeral in `Fin 5` is read modulo 5, so `7` is accepted and means 2, and nothing checks that 7 is not a cell of R_5. Part II writes cells with their proofs for this reason.

A function whose output type depends on its input is a *dependent function*. `last n` is the largest number below $n + 1$, and its type

`Fin (n + 1)` changes with n. Its proof, `by omega`, asks Lean to show $n < n + 1$ by arithmetic.

```
def last (n : Nat) : Fin (n + 1) := ⟨n, by omega⟩
#check last  -- last (n : Nat) : Fin (n + 1)
def p : (n : Nat) × Fin n := ⟨3, ⟨1, by decide⟩⟩
```

The type of `last` is written `(n : Nat) → Fin (n + 1)`: it is the book's $\Pi_{n:\mathbb{N}}\mathsf{Fin}(n+1)$. In a *dependent pair* the type of the second entry depends on the first: a value of `(n : Nat) × Fin n` is a number n together with a number below it, the book's $\Sigma_{n:\mathbb{N}}\mathsf{Fin}(n)$. The pair `p` is 3 with 1.

Propositions are types

A proposition is a type as well, of the sort `Prop`, and a proof of it is a value of that type. The equation `2 + 3 = 5` is a proposition, and `rfl` is a proof of it, because both sides compute to 5. `example` states a proposition and gives its proof; `theorem` does the same and gives the proof a name.

```
#check (2 + 3 = 5 : Prop)  -- 2 + 3 = 5 : Prop
example : 2 + 3 = 5 := rfl
```

A false proposition has no proof. Asked to decide one by computing, Lean answers that it is false:

```
example : 2 + 2 = 5 := by decide
```

Lean refuses it:

```
Tactic ‘decide‘ proved that the proposition
  2 + 2 = 5
is false
```

The logical connectives are types built from types. A proof of an implication $p \to q$ is a function that turns any proof of p into a proof of q. A proof of $p \wedge q$ is a pair of proofs. A proof of $\exists x,\ P(x)$ is a pair: a witness x together with a proof of $P(x)$. A proof of $\forall x,\ P(x)$ is a dependent function, giving a proof of $P(x)$ for every x. `False` is the proposition with no proof, and $\neg p$ means $p \to$ `False`.

```
example (p : Prop) : p→p := fun h => h
example (p q : Prop) (hp : p) (hq : q) : p ∧ q := ⟨hp, hq⟩
example : ∃ n : Nat, n * n = 9 := ⟨3, rfl⟩
```

The first line proves $p \to p$ by the function that returns its input. The second pairs two proofs. The third gives the witness 3 with the proof that $3 \cdot 3 = 9$. A proof of an existence statement is a witness with its evidence, the shape of every witness in the book.

Tactics

A long proof is easier to build in steps. After `by` come *tactics*: commands that each change the *goal*, the statement still to be proved, until nothing is left to prove. In the editor the infoview shows the goal after each step. These are the tactics this text uses:

Tactic	What it does
`rfl`	proves an equation whose two sides compute to one value
`decide`	checks a statement about finitely many cases by computing each
`omega`	proves facts of integer arithmetic: $+$, $-$, $\leq$, $<$, and products, quotients and remainders by fixed numbers
`intro h`	begins a proof of $p \to q$ by assuming p and naming its proof `h`
`exact e`	finishes the goal with the proof `e`
`cases`, `induction`	split the goal by the constructors of a value
`simp`	rewrites the goal with definitions and known equations
`grind`	searches for a proof by combining the facts at hand

```
example (p q : Prop) : p → (p → q) → q := by
  intro hp hpq
  exact hpq hp
example (a b : Nat) (h : a < b) : a + 1 ≤ b := by omega
example : ∀ n : Fin 10, n.val * n.val % 4 ≠ 2 := by decide
```

The first proof assumes p and $p \to q$, then applies the second to the first. The second is arithmetic. The third checks, for the ten numbers below 10, that a square leaves remainder 0 or 1 on division by 4, never 2.

Induction proves a statement for every natural number from two cases, 0 and $n + 1$; in the second case the statement for n may be used. Here it proves Gauss's formula $2(1 + 2 + \cdots + n) = n(n + 1)$:

```
theorem sumTo_formula (n : Nat) : 2 * sumTo n = n * (n + 1) := by
  induction n with
  | zero      => rfl
  | succ n ih => simp only [sumTo]; grind
```

In the case `succ n` the name `ih` stands for the formula at n. `simp only [sumTo]` unfolds `sumTo` once, at $n+1$, and `grind` finishes the algebra.

Arguments that Lean finds

An input in square brackets is supplied by Lean, not by the caller. `[Add α]` asks for an addition on the type α. Lean has one for `Nat`, `Int` and `Rat`, so one definition serves all three:

```
def sum3 {α : Type} [Add α] (a b c : α) : α := a + b + c
#eval sum3 1 2 3                    -- 6
#eval sum3 (1/2 : Rat) (1/3) (1/6)  -- 1
```

Such an input is called an *instance*, and the family of instances it belongs to a *class*. `decide` works in the same way: it asks for an instance that says how to decide the statement, and Lean comes with instances for numbers, for the types `Fin n`, and for statements built from them. Part II adds one class of its own, `Listed`, for the finite types of Chapter 1.

Reading Lean's messages

The commonest message is a type mismatch. It shows an expression, the type it has, and the type Lean expected there:

```
example : double 2 = "four" := rfl
```

Lean refuses it:

```
Type mismatch
  "four"
has type
  String
but is expected to have type
  Nat
```

Read it from the bottom. Lean expected a `Nat` on the right of `=`, because the left side, `double 2`, is a `Nat`, and it found a `String`. A misspelt name is reported as unknown:

```
#eval doubel 21
```

Lean refuses it:

```
Unknown identifier ‘doubel‘
```

When a message is hard to read, `#check` on the parts of the expression usually shows which type is not the one meant.

Further reading

Avigad, de Moura, Kong and Ullrich teach Lean from its types and tactics, in the order of this part and much further [1]. Christiansen teaches Lean as a programming language [3]. Avigad and Massot teach mathematics in Lean with Mathlib [2].

Part II. Chapter 1 of the book in Lean

This part writes the constructions of Chapter 1 of the book in Lean, in the order of the chapter. Lean's type theory contains the types and rules of Section 1.11 [6], and Lean checks every judgement a program makes. A formation line becomes a declaration, the inhabitant a plate works through becomes a value, and a value of the wrong type is refused with a message.

The listings are cut from one file, which Lean 4.34 accepts as a whole; it uses Lean's core language and no library. A routine part of a value or a proof is printed as $\cdots$, and the file contains it.

From the book's notation to Lean

Table 1 sets the reading card of Section 1.4 beside its Lean spelling.

Section 1.4 read $a : A$ as $a \in A$ for the finite cell types of this chapter. Lean is given that reading by a list of the values:

```
class Listed (A : Type) where
  elems    : List A                -- the values of A
  complete : ∀ a, a ∈ elems        -- and every value is among them
```

With the list, `decide` checks a statement about every value of the type by running through the values. The file gives such lists for the cells of rods, the tags, and pairs and tagged pairs of listed types.

Book	Lean	Read it as
$a : A$	`a : A`	a is a value of A
$A \to B, \quad a \mapsto b$	`A → B, fun a => b`	functions
$\Sigma_{i:I} B(i), \quad (i, b)$	`(i : I) × B i, ⟨i, b⟩`	a tagged choice
$\Pi_{i:I} B(i)$	`(i : I) → B i`	a complete assignment
data $\Longrightarrow$ $C(\text{data}) : T$	`def C (data) : T := ...`	a formation line
introduction rules	`inductive T where | c : ...`	one line per rule
elimination rule	a definition by cases	one case per rule
$a \equiv b$	`rfl : a = b`	equal by computation
$\mathrm{Id}_A(a, b)$	`a = b`	the identity type
$\{s \mid \mathsf{Legal}(s)\}$	`{ s // Legal s }`	a table with its legality

Table 1: The reading card in Lean. Lean writes a tagged choice with $\times$ because, over a constant family, it is the product (Section 1.4).

Rods: their rules and their laws

Section 1.11 gave rods two introduction rules. In Lean they are an inductive type, one line for each rule, and uniform substitution (Section 1.2) is a third rule. The tags of a gluing are a type of two values.

```
inductive Tag | L | R
```

```
inductive OrderedForm where
  | rod   : Nat → OrderedForm
  | glue  : OrderedForm → OrderedForm → OrderedForm
  | subst : OrderedForm → OrderedForm → OrderedForm
```

The elimination rule is definition by cases: say what a definition gives for each rule, given what it gives for the rods the rule takes. The cells are defined so. A cell of R_n is a number below n, a cell of $\mathsf{Glue}(P, Q)$ is a tag with a cell of the rod the tag names, and a cell of $\mathsf{Substitute}(P, Q)$ is a pair.

```
def cells : OrderedForm → Type
  | rod n      => Fin n
  | glue P Q   => (t : Tag) × match t with
                              | .L => cells P
                              | .R => cells Q
  | subst P Q => cells P × cells Q
```

Size, position and order are defined by the same cases. The second and third lines of `pos` are the position equations of Section 1.2, and the last line is an instance of the computation rules of Section 1.11: `rfl` checks it for every P and Q, with no induction.

```
def size : OrderedForm → Nat
  | rod n      => n
  | glue P Q   => size P + size Q
  | subst P Q => size P * size Q
def pos : (X : OrderedForm) → X.cells → Nat
  | rod _,      i        => i
  | glue P _,   ⟨.L, p⟩  => pos P p
  | glue P Q,   ⟨.R, q⟩  => size P + pos Q q
  | subst P Q, (p, q)    => size Q * pos P p + pos Q q
def before (X : OrderedForm) (a b : X.cells) : Prop := X.pos a < X.pos b
example (P Q : OrderedForm) : (glue P Q).size = P.size + Q.size := rfl
```

Proposition 1.31 says that the order on the cells of every rod is linear. In Lean two cells with one position are one cell, and linearity follows from the order of the numbers:

```
theorem pos_inj : ∀ (X : OrderedForm) (a b : X.cells),
    X.pos a = X.pos b → a = b := · ·
theorem linear (X : OrderedForm) (a b : X.cells) :
    a ≠ b → X.before a b ∨ X.before b a := fun ne => · ·
```

The proof of `pos_inj` takes the rules in pairs, as the proof of proposition 1.31 did. For substitution it divides a position by the size of the inner rod to recover the outer cell.

Witnesses, and three equalities in $2 + 3 = 5$

A perfect alignment is a structure with five fields: a map, a map back, two equations saying that each undoes the other, and the law that the order is kept. The last three fields are proofs, so a value must supply them.

```
structure Align (P Q : OrderedForm) where
  map        : P.cells → Q.cells
  back       : Q.cells → P.cells
  back_map   : ∀ x, back (map x) = x
  map_back   : ∀ y, map (back y) = y
  keeps_order : ∀ x y, P.before x y → Q.before (map x) (map y)
```

The alignment of Figure 1.9 is written with its five arrows, in two lines for the two tags. `decide` checks the three laws on the five cells and on the twenty-five pairs of cells. A wrong arrow would fail a check, and Lean would refuse the value.

```
def α : Align (glue (rod 2) (rod 3)) (rod 5) where
  map
    | ⟨.L, i⟩ => ⟨i, by omega⟩          -- L0↦0, L1↦1
    | ⟨.R, j⟩ => ⟨2 + j, by omega⟩     -- R0↦2, R1↦3, R2↦4
  back i := if h : i < 2 then ⟨.L, ⟨i, h⟩⟩
            else ⟨.R, ⟨i - 2, by omega⟩⟩
  back_map    := by decide
  map_back    := by decide
  keeps_order := by decide
```

The three equalities of Section 1.11 are three declarations:

```
example : 2 + 3 = 5 := rfl                          -- a computation
def isGlued : OrderedForm→Prop
  | glue _ _ => True
  | _        => False
example : glue (rod 2) (rod 3) ≠ rod 5 :=           -- an empty type
  fun h => (h ▶ trivial : isGlued (rod 5))
example : Align (glue (rod 2) (rod 3)) (rod 5) := α  -- one value
```

The second is the test of Section 1.11. `isGlued` is true of every glued rod and false of every R_n. A proof `h` that the glued rod equals R_5 would turn `trivial`, the proof that the glued rod is glued, into a proof that R_5 is glued, which is `False`. So there is no such `h`.

An empty type is stated as a function into `False`: from a value it would return a proof of what has none. The rods R_2 and R_3 have no alignment (Section 1.4). The map back would send three cells into two, two of them would meet, and the map would then send one cell to two different cells. The order witness of Section 1.3 is a tagged choice whose tag is a rod: a residual C, with an alignment of $\mathsf{Glue}(R_m, C)$ with R_n. The evidence that $3 \le 5$ has the residual R_2, and its alignment is written as α was.

```
theorem no_align (a : Align (rod 2) (rod 3)) : False := ⋯
def Fits (m n : Nat) : Type :=
  (C : OrderedForm) × Align (glue (rod m) C) (rod n)
def threeInFive : Fits 3 5 :=
  ⟨rod 2, ⋯⟩
```

Laws as scripts

Each move in the table of laws of Section 1.3 is a function on cells. The exchange swaps the tags. The transpose swaps the coordinates, and transposing back is the identity by computation. The distributive

cut sends a cell of the substituted segmented rod to the block its tag names, in the same row. Regrouping is written as the exchange is. Lean refuses a definition by cases that leaves a case out, so every move is total.

```
def exchange (P Q : OrderedForm) :
    (glue P Q).cells → (glue Q P).cells
  | ⟨.L, p⟩ => ⟨.R, p⟩
  | ⟨.R, q⟩ => ⟨.L, q⟩
def transpose (m n : Nat) :
    (subst (rod m) (rod n)).cells → (subst (rod n) (rod m)).cells
  | (i, j) => (j, i)
example (m n : Nat) : transpose n m ∘ transpose m n = id := rfl
def cut (P Q₁ Q₂ : OrderedForm) :
    (subst P (glue Q₁ Q₂)).cells → (glue (subst P Q₁) (subst P Q₂)).cells
  | (p, ⟨.L, q⟩) => ⟨.L, (p, q)⟩
  | (p, ⟨.R, q⟩) => ⟨.R, (p, q)⟩
```

A tagged choice and a complete assignment

The family F of Section 1.4 is a function from tags to rods. A value of the tagged choice is a tag with a cell of the rod it names, and the list of values has five entries, none repeated:

```
def F : Tag → OrderedForm
  | .L => rod 2
  | .R => rod 3
example : (t : Tag) × (F t).cells := ⟨.R, 1⟩          -- the cell R1
def values (A : Type) [Listed A] : List A := Listed.elems
example : (values ((t : Tag) × (F t).cells)).length = 5 := rfl
example : (values ((t : Tag) × (F t).cells)).Nodup := by decide
```

Lean refuses the pair $(L, 2)$:

```
example : (t : Tag) × (F t).cells := ⟨.L, ⟨2, by decide⟩⟩
```

Lean refuses it:

```
Tactic ‘decide‘ proved that the proposition
  2 < 2
is false
```

Written $\langle 2, h \rangle$, the cell carries a proof h that $2 < 2$, and there is none. A bare numeral would not be checked: Lean reads a numeral in `Fin n` modulo n, so that $\langle L, 2 \rangle$ would be the cell $(L, 0)$.

A complete assignment is a function defined at both tags. It corresponds to the pairs of a row value and a column value (Section 1.4) by two functions, and the two examples after them are the round trips. The first holds by computation. The second compares functions and uses `funext`: functions that agree at every input are equal. Lean proves this principle from its quotient types; an intensional type theory without them must assume it.

```
def f : (t : Tag) → (F t).cells                    -- [L↦0, R↦1]
  | .L => 0
  | .R => 1
def toPair (g : (t : Tag) → (F t).cells) : Fin 2 × Fin 3 := (g .L, g .R)
def ofPair : Fin 2 × Fin 3 → (t : Tag) → (F t).cells
  | (i, _), .L => i
  | (_, j), .R => j
example (x : Fin 2 × Fin 3) : toPair (ofPair x) = x := rfl
example (g : (t : Tag) → (F t).cells) : ofPair (toPair g) = g :=
  funext fun t => by cases t <;> rfl
example : (values (Fin 2 × Fin 3)).length = 6 := rfl
```

Formation lines

A formation line becomes the head of a declaration: its inputs are the parameters, and the constructed type follows the colon. On cells, the assembly of Section 1.4 is the tagged choice itself. The code and the history of a form (Figure 1.21) are a tagged choice whose tag is the

code. `History k` holds the construction trees of k cells: an inductive family, with one type for each code. The two trees of Figure 1.22 are two values over the code 3.

```
def assemble (I : Type) (F : I→Type) : Type := (i : I) × F i
inductive History : Nat→Type
  | atom  : Char→History 1
  | glue  : History m→History n→History (m + n)
  | subst : History m→History n→History (m * n)
def AFForm : Type := (k : Nat) × History k
def treeA : AFForm :=
  ⟨3, .glue (.atom 'a')
        (.subst (.glue (.atom 'b') (.atom 'c')) (.atom 'd'))⟩
def treeB : AFForm :=
  ⟨3, .glue (.atom 'a') (.glue (.atom 'd') (.atom 'd'))⟩
```

A tree of four cells is a history of the four-cell code and of no other:

```
def ab : History 2 := .glue (.atom 'a') (.atom 'b')
example : AFForm := ⟨3, .glue ab ab⟩
```

Lean refuses it:

```
Application type mismatch: The argument
  ab.glue ab
has type
  History (2 + 2)
but is expected to have type
  History 3
in the application
  ⟨3, ab.glue ab⟩
```

The base cut of a pointed wheel is a cell of the wheel's own form, so the type of the second field mentions the first. The arc state and the gapped state of Figure 1.15 are two tables with one tally. Normalization takes its evidence as an input, and so cannot be applied to R_2 and R_3.

```
structure Wheel where
  F    : OrderedForm
  base : F.cells                        -- the cut: a cell of F itself
def eight : Wheel := ⟨rod 8, 0⟩
def tally (s : Fin 8→Bool) : Nat :=
  ((List.finRange 8).filter (s ·)).length
def arc    : Fin 8→Bool := fun i => i.val ∈ [0, 1, 2]
def gapped : Fin 8→Bool := fun i => i.val ∈ [1, 4, 6]
example : tally arc = 3 ∧ tally gapped = 3 ∧ arc ≠ gapped := by decide
def normalize (X : OrderedForm) (n : Nat) (_ : Align X (rod n)) :
    OrderedForm :=
  rod n
example : OrderedForm := normalize (glue (rod 2) (rod 3)) 5 α
```

Time

A finite temporal support has positions, a successor defined on some of them, and the proof that the successor is injective. The line and the cycle supply that proof once for every n, and a clock is a pointed cycle.

```
structure TimeForm where
  n    : Nat
  next : Fin n→Option (Fin n)           -- none: no successor
  inj  : ∀ s t u, next s = some u→next t = some u→s = t
def LinearTime (n : Nat) : TimeForm where
  n := n
  next t := if h : t + 1 < n then some ⟨t + 1, h⟩ else none
  inj s t u h₁ h₂ := · ·
def CycleTime (n : Nat) : TimeForm where
  n := n
  next t := some ⟨(t + 1) % n, Nat.mod_lt _ (by have := t.isLt; omega)⟩
  inj s t u h₁ h₂ := · ·
structure PointedTimeForm where
  T  : TimeForm
  t₀ : Fin T.n
def PointTime (T : TimeForm) (t₀ : Fin T.n) : PointedTimeForm := ⟨T, t₀⟩
```

```
def forgetPoint (P : PointedTimeForm) : TimeForm := P.T
def Clock (L : Nat) (h : 0 < L) : PointedTimeForm :=
  PointTime (CycleTime L) ⟨0, h⟩
```

A rectangular tapestry (definition 1.5) keeps its support, its ordered rows, its temporal palette and one thread for every row. Lean numbers the rows of the tetractys palette from 0, so row r has $r + 1$ phases. Each thread of Figure 1.25 reads a tick modulo the number of phases of its row, and the names $1, \ldots, 10$ are computed for the reader. The two outlined columns, the distinctness of the twelve columns and the ten values of the palette are checked:

```
structure RectTapestry where
  T : TimeForm
  R : OrderedForm                          -- the rows, in order
  V : R.cells→Type                         -- the temporal palette
  e : (r : R.cells)→Fin T.n→V r            -- one thread for every row
def thread (r : Fin 4) (t : Fin 12) : Fin (r + 1) :=
  ⟨t % (r + 1), Nat.mod_lt _ (by omega)⟩
def recurrence : RectTapestry :=
  ⟨CycleTime 12, rod 4, fun r => Fin (r + 1), thread⟩
def name (r : Fin 4) (j : Fin (r + 1)) : Nat := 1 + (r + 1) * r / 2 + j
def column (t : Fin 12) : List Nat :=
  (List.finRange 4).map fun r => name r (thread r t)
example : column 0 = [1, 2, 4, 7] ∧ column 5 = [1, 3, 6, 8] := by decide
example : ((List.finRange 12).map column).Nodup := by decide
example : (values ((r : Fin 4) × Fin (r + 1))).length = 10 := rfl
```

A laying sends the number of phases of a row and a tick to a phase. The period of a row is the least step after which it repeats, and the spectra of proposition 1.10 are computed:

```
def Laying := Nat→Nat→Nat          -- phases k and tick t↦ a phase
def layRec : Laying := fun k t => t % k
def layDiv : Laying := fun k t => k * t / 12
def period (row : Nat→Nat) : Nat :=
  ((List.range 12).map (· + 1)).find? (fun d => 12 % d == 0 &&
```

```
      (List.range 12).all fun t => row ((t + d) % 12) == row t)
    |>.getD 12
def spectrum (ℓ : Laying) : List Nat :=
  [1, 2, 3, 4].map fun k => period (ℓ k)
example : spectrum layRec = [1, 2, 3, 4]    := by decide
example : spectrum layDiv = [1, 12, 12, 12] := by decide
```

An occurrence history (definition 1.6) is a path in a transition graph: an inductive family indexed by its two ends and its number of transitions. A tick realization (definition 1.7) keeps its certificate as a field, so no value can be formed without it. Every certified record of Section 1.5 has this shape.

```
structure Graph where
  State : Type
  Move  : State→State→Type
inductive Path (G : Graph) : G.State→G.State→Nat→Type
  | stay : Path G x x 0
  | step : G.Move x y→Path G y z H→Path G x z (H + 1)
structure TickRealization (G : Graph) (T : PointedTimeForm) where
  q     : G.State→Fin T.T.n                    -- phase projection
  x₀    : G.State
  κ     : {x y : G.State}→G.Move x y→Bool      -- registers a tick?
  start : q x₀ = T.t₀
  law   : ∀ {x y} (a : G.Move x y),
            if κ a then T.T.next (q x) = some (q y) else q y = q x
```

The digits of a cyclarchy (definition 1.8) are a complete assignment over a family that varies, one digit for each radix, written as nested pairs. Adding one carries into the next digit, and flattening makes the cyclarchy $(3, 4)$ a clock of twelve:

```
def Digits : List Nat→Type
  | []      => Unit
  | N :: Ns => Fin N × Digits Ns
def addCarry : (Ns : List Nat)→Digits Ns→Bool→Digits Ns
  | [],      (),     _ => ()
  | N :: Ns, (a, r), c =>
```

```
      if c then
        if h : a + 1 < N then (⟨a + 1, h⟩, r)
        else (⟨0, by omega⟩, addCarry Ns r true)   -- the carry
      else (a, r)
def inc (Ns : List Nat) (d : Digits Ns) : Digits Ns := addCarry Ns d true
def flatten : (Ns : List Nat) → Digits Ns → Nat
  | [],      ()     => 0
  | N :: Ns, (a, r) => a + N * flatten Ns r
example : inc [3, 4] (⟨2, by omega⟩, ⟨0, by omega⟩, ())
          = (⟨0, by omega⟩, ⟨1, by omega⟩, ()) := by decide
example : ∀ d : Digits [3, 4],
    flatten [3, 4] (inc [3, 4] d) = (flatten [3, 4] d + 1) % 12 := by
  decide
```

The covering and grouping equations of Section 1.5 are two statements about a laid row, and the division laying fails the first:

```
def covering (L k : Nat) (c : Nat → Nat) : Prop :=
  ∀ t < L, c ((t + 1) % L) = (c t + 1) % k
def grouping (L k : Nat) (p : Nat → Nat) : Prop :=
  ∀ t < L, p ((t + L / k) % L) = (p t + 1) % k
example : covering 12 4 (layRec 4) ∧ grouping 12 4 (layDiv 4) := by decide
example : ¬ covering 12 4 (layDiv 4) := by decide
```

The calibration of Figure 1.38, the gear phases, the event tapestry and the duration record are declared in the same way, and the closing table of this part names them. The calibration record carries as its certificate a proof that every pair of readings meets the law, and the file computes from it the elapsed interval $[2999, 3001]U$.

Meaning

An interpretation profile declares its endpoints, its realizers and its law. An interpretation witness (definition 1.9) is a realizer with its certificate. The semantic fibre is a tagged choice over the profiles in use, and the selected layer a further one.

```
structure Profile (Src : Type) where
  End      : Type                       -- what a meaning may produce
  Realizer : Type                       -- the rules that may produce it
  produces : Realizer→Src→End→Prop
  law      : Realizer→Prop
structure IntWit (C : Profile Src) (F : Src) (θ : C.End) where
  realizer : C.Realizer
  cert     : C.law realizer ∧ C.produces realizer F θ
def Sem (Γ : I→Profile Src) (F : Src) : Type :=
  (i : I) × (θ : (Γ i).End) × IntWit (Γ i) F θ
def PointedInt (Γ : I→Profile Src) : Type := (F : Src) × Sem Γ F
```

The two disciplines of the game of Section 1.7 are two profiles. A law asks for its equation and for the phase typing of the rows; a profile asks that the frame be what the laying produces, with the declared spectrum. The two moves are two witnesses, and the moves side by side are two values of the fibre. The move that was not made is an empty type: no laying at all strings the palette into the division frame under the law of the clock. The frame shows the row of two unchanged from tick 0 to tick 1, and the covering equation requires a change.

```
def Palette := List Nat                  -- the row sizes of the theme
def P : Palette := [1, 2, 3, 4]
def Strung := Fin 4→Fin 12→Nat           -- four wires, twelve rungs
def lay (ℓ : Laying) (F : Palette) : Strung := fun r t => ℓ (F.getD r 0) t
def clockLaw (ℓ : Laying) : Prop :=
  ∀ k ∈ [1, 2, 3, 4], covering 12 k (ℓ k) ∧ ∀ t < 12, ℓ k t < k
def stringLaw (ℓ : Laying) : Prop :=
  ∀ k ∈ [1, 2, 3, 4], grouping 12 k (ℓ k) ∧ ∀ t < 12, ℓ k t < k
def profile (law : Laying→Prop) (spec : List Nat) :
    Profile Palette where
  End      := Strung
  Realizer := Laying
  produces ℓ F θ := θ = lay ℓ F ∧ spectrum ℓ = spec
  law      := law
def Crec := profile clockLaw  [1, 2, 3, 4]
```

```
def Cdiv := profile stringLaw [1, 12, 12, 12]
def wRec : IntWit Crec P (lay layRec P) :=
  ⟨layRec, (by decide : clockLaw layRec), rfl, by decide⟩
def wDiv : IntWit Cdiv P (lay layDiv P) :=
  ⟨layDiv, (by decide : stringLaw layDiv), rfl, by decide⟩
inductive Discipline | clock | string
def Γ : Discipline→Profile Palette
  | .clock  => Crec
  | .string => Cdiv
def moves : List (Sem Γ P) := [⟨.clock, _, wRec⟩, ⟨.string, _, wDiv⟩]
theorem no_move : IntWit Crec P (lay layDiv P)→False := · ·
```

Spaces and states

A finite cell pattern (definition 1.11) declares cells, their dimensions, the immediate faces and a face-closed boundary, with the two laws as fields. SquareGrid(m, n) gives the cells of each dimension.

```
structure Canvas where
  Cell     : Type
  dim      : Cell→Nat
  face     : Cell→Cell→Prop        -- a is an immediate face of b
  raises   : ∀ a b, face a b→dim b = dim a + 1
  boundary : Cell→Prop              -- the declared outer boundary
  closed   : ∀ a b, face a b→boundary b→boundary a
inductive Dim | vertex | edge | face
def SquareGrid (m n : Nat) : Dim→Type
  | .vertex => Fin (m + 1) × Fin (n + 1)
  | .edge   => Fin m × Fin (n + 1) ⊕ Fin (m + 1) × Fin n
  | .face   => Fin m × Fin n
```

Listing the cells of the board gives the counts of Section 1.8. The colouring rule, the shared edge and the domino of Figure 1.48 follow. The placement at $d6$ and $e7$ has no value, since its field would prove that `false = true`; `nofun`, the function with no cases, shows it.

```
def count (A : Type) [Listed A] : Nat := (values A).length
example : count (SquareGrid 8 8 .vertex) = 81  := rfl
example : count (SquareGrid 8 8 .edge)   = 144 := rfl
example : count (SquareGrid 8 8 .face)   = 64  := rfl
def Square := Fin 8 × Fin 8                -- (file, rank): a1 is (0, 0)
def dark (s : Square) : Bool := (s.1.val + s.2.val) % 2 == 0
def shareEdge (s t : Square) : Bool :=
  (s.1 == t.1 && (s.2.val + 1 == t.2.val || t.2.val + 1 == s.2.val)) ||
  (s.2 == t.2 && (s.1.val + 1 == t.1.val || t.1.val + 1 == s.1.val))
structure Domino (a b : Square) : Prop where
  link : shareEdge a b = true
example : Domino (6, 6) (7, 6) := ⟨rfl⟩          -- g7 and h7
example : Domino (3, 5) (4, 6)→False := nofun    -- d6 and e7
```

The state space is a subtype. Lean treats any two proofs of one proposition as equal, so two states with one table are equal by computation. This is legality read as a property (Section 1.11): Lean counts tables, not proofs.

```
def Table := Square→Fin 13              -- six white, six black, empty
def State (Legal : Table→Prop) : Type := { s : Table // Legal s }
example (Legal : Table→Prop) (s : Table) (p q : Legal s) :
    (⟨s, p⟩ : State Legal) = ⟨s, q⟩ := rfl
```

Scripts and roles

A script retains its operations as well as its action. The transpose followed by itself acts as the identity and records two operations, as Section 1.3 said of the two-step script. Positions with White to play and positions with Black to play are two types, and a White turn goes from the first to the second (Figure 1.57).

```
structure Script (A B : Type) where
  ops : List String                    -- the operations it retains
  act : A→B                            -- its action on states
def Script.comp (g : Script B C) (f : Script A B) : Script A C :=
  ⟨f.ops ++ g.ops, g.act ∘ f.act⟩
def τ (m n : Nat) : Script (Fin m × Fin n) (Fin n × Fin m) :=
  ⟨["transpose"], fun (i, j) => (j, i)⟩
example : ((τ 3 2).comp (τ 2 3)).act = id := rfl
example : ((τ 3 2).comp (τ 2 3)).ops = ["transpose", "transpose"] := rfl
structure WhiteToPlay where board : Table
structure BlackToPlay where board : Table
def turnW (move : Table→Table) : Script WhiteToPlay BlackToPlay :=
  ⟨["White's move"], fun s => ⟨move s.board⟩⟩
```

Two White turns do not compose:

```
example (m : Table→Table) : Script WhiteToPlay BlackToPlay :=
  (turnW m).comp (turnW m)
```

Lean refuses it:

```
Application type mismatch: The last
  turnW m
argument has type
  Script WhiteToPlay BlackToPlay
but is expected to have type
  Script WhiteToPlay WhiteToPlay
in the application
  (turnW m).comp (turnW m)
```

`#eval` prints a value for the reader, and no later declaration can refer to what it printed. A definition is a value that later declarations may use.

```
def lit : Fin 8→Bool := arc          -- derived data: a rule may read it
#eval tally lit                      -- prints 3, a reading: no rule can
```

What Lean does not see

Lean checks that each value has its type. As Section 1.11 said of the rules, it does not say what a value is for. The material balance of a position could be computed by one definition and consumed by the next, and Lean would accept both. The rule of Section 1.9 that nothing may depend on a reading is kept by whoever writes the program, by leaving readings in `#eval`.

Nor does Lean see pictures. Section 1.4 specified a structure by a picture and stated its laws in the text. In Lean the inductive type is the specification, and a law the picture shows is a theorem, as `pos_inj` is for the row of cells.

Every formation line in Lean

Each formation line of this chapter, and the declaration of the file that writes it:

Formation line	Lean
$\mathsf{Glue}(P, Q)$, R_n, $\mathsf{Substitute}(P, Q)$	`OrderedForm`, three rules
$\mathsf{Assemble}(I; F)$, $\mathsf{Index}(n)$, $\varnothing$, an atom	`assemble`, `Fin n`, `Fin 0`, `Fin 1`
code k and history $h : \mathsf{History}(k)$	`AFForm`, `History`
$\mathsf{Wheel}_n(F; b)$, $\mathsf{Normalize}(X; \alpha)$	`Wheel`, `normalize`
$\mathsf{LinearTime}(n)$, $\mathsf{CycleTime}(n)$, $\mathsf{PointTime}(T; t_0)$	`LinearTime`, `CycleTime`, `PointTime`
$\mathsf{toClock}(W; i)$, SampleClock	`toClock`, `SampleClock`
$\mathsf{RectTapestry}(T; R, V, e)$	`RectTapestry`
$\mathsf{OccurrenceHistory}(G; \gamma)$	`Path`

Formation line	Lean
$\mathsf{TickRealization}(G, T)$	`TickRealization`
$\mathsf{Cyclarchy}(N)$	`Digits`, `inc`, `flatten`
$\mathsf{EventTapestry}(T; D)$	`EventData`
$\mathsf{DurationRecord}(\gamma; U, d)$	`DurationRecord`, `D`
$\mathsf{EventAssociation}(E, \gamma)$	`EventAssociation`
$\mathsf{CalibrationRecord}(R; \rho, \epsilon, c)$, $\mathsf{Elapsed}(w; e, f)$	`CalibrationRecord`, `elapsed`
$\mathsf{IntWit}_{\mathcal{C}}(F, \theta)$, $\mathsf{Sem}_{\Gamma}(F)$, $\mathsf{PointedInt}_{\Gamma}$	`IntWit`, `Sem`, `PointedInt`
$\mathsf{SquareGrid}(m, n)$, $\mathsf{Canvas}(F; c)$	`SquareGrid`, `Canvas`
$\mathsf{AttachCell}(X, c; w)$	`attachCell`, `PatchWitness`
$\mathsf{State}(A)$, a frame and its instances	`State`, `Frame`, `Frame.Instance`
$\mathsf{Refine}(X; \rho)$	`RefinementData`
$g \circ f : \mathsf{Script}(A, C)$	`Script.comp`

The file also counts the eight symmetries of the square of Section 1.8 from its incidence.

Connections and sources

Lean 4 is described by de Moura and Ullrich. Its type theory goes back to the calculus of constructions of Coquand and Huet, with the inductive types of Coquand and Paulin; inductive families, such as the histories and paths of this part, are Dybjer's. *Sources:* [6, 4, 5, 7].

Bibliography

[1] Jeremy Avigad, Leonardo de Moura, Soonho Kong, and Sebastian Ullrich. Theorem proving in Lean 4. https://lean-lang.org/theorem_proving_in_lean4/. With contributions from the Lean Community; consulted 2026-10-08.

[2] Jeremy Avigad and Patrick Massot. Mathematics in Lean. https://leanprover-community.github.io/mathematics_in_lean/. Consulted 2026-10-09.

[3] David Thrane Christiansen. Functional programming in Lean. https://lean-lang.org/functional_programming_in_lean/. Consulted 2026-10-09.

[4] Thierry Coquand and Gérard Huet. The calculus of constructions. *Information and Computation*, 76(2–3):95–120, 1988.

[5] Thierry Coquand and Christine Paulin. Inductively defined types. In *COLOG-88*, Lecture Notes in Computer Science, pages 50–66. Springer, 1990.

[6] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, Lecture Notes in Computer Science, pages 625–635. Springer, 2021.

[7] Peter Dybjer. Inductive families. *Formal Aspects of Computing*, 6(4):440–465, 1994.